

CAPÍTULO 05 · GUÍA DEL ESTUDIANTE

Spring Boot + PostgreSQL + JPA/Hibernate · Persistencia real sobre tu proyecto asignado

PROGRAMACIÓN APLICADA 2026-2 · Backend ParkFlow 360

1. Qué vas a lograr

Hasta el capítulo anterior tu API podía trabajar con datos temporales en memoria. Desde este capítulo comenzarás a guardar y consultar información en la base PostgreSQL que tu grupo ya diseñó. Seguirás el ejemplo del profesor con Cliente → Vehículo, pero debes trabajar con las dos tablas 1:N que seleccionaste en tu propio proyecto.

No copies ParkFlow

Cliente y Vehículo son el proyecto espejo del profesor. Tú debes usar tu entidad padre y tu entidad dependiente, con los nombres, columnas, restricciones y schema reales de tu proyecto.

2. Traducción obligatoria profesor → tu proyecto

Profesor	Tu proyecto
Base parkflow360	Tu base de datos
Schema parkflow	Tu schema
cliente	Tu tabla padre 1:N
vehiculo	Tu tabla dependiente
cliente_id	La FK de tu tabla dependiente
ClienteJpaEntity	<EntidadPadre>JpaEntity
SpringDataClienteRepository	SpringData<EntidadPadre>Repository

3. Antes de programar: completa esta ficha

Dato	Tu respuesta
Nombre del proyecto	_____
Base de datos	_____
Schema	_____
Tabla padre	_____
PK de la tabla padre	_____
Tabla dependiente	_____
FK hacia la tabla padre	_____
UNIQUE importantes	_____

4. Comprueba tu base desde DataGrip

```
SELECT current_database() AS base_actual,  
       current_schema()  AS schema_actual,  
       current_user       AS usuario_actual;
```

Si la base no es la de tu proyecto, NO continúes. La configuración correcta en DataGrip no garantiza que Spring Boot esté configurado igual; ambas conexiones se verificarán por separado.

5. Conceptos mínimos que debes poder explicar

Concepto	Significado
JDBC	API Java de bajo nivel para comunicarse con bases relacionales.
JPA	Especificación estándar para mapear objetos y tablas.
Hibernate	Implementación ORM utilizada por Spring Boot.
Spring Data JPA	Abstracción de repositorios sobre JPA/Hibernate.
ORM	Mapeo entre objetos Java y estructuras relacionales.

6. Agrega las dependencias necesarias

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

7. Configura application.yml con TU base y TU schema

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/TU_BASE
    username: TU_USUARIO
    password: TU_PASSWORD

  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        default_schema: TU_SCHEMA
        format_sql: true
    open-in-view: false

logging:
  level:
    org.hibernate.SQL: debug
```

Regla obligatoria

Usa ddl-auto=validate. No uses create, create-drop ni update para “hacer que funcione”. Tu grupo ya diseñó la base físicamente; Java debe adaptarse a ese contrato.

8. Estructura de paquetes que debes crear

```
src/main/java/com/<tu-proyecto>
├── <TuAplicacion>.java
├── <modulo-padre>
│   ├── domain
│   │   └── model
│   │       └── <EntidadPadre>.java
│   ├── infrastructure
│   │   ├── adapter
│   │   │   └── out
│   │   │       └── persistence
│   │   │           ├── entity
│   │   │           │   └── <EntidadPadre>JpaEntity.java
│   │   │           ├── mapper
│   │   │           │   └── <EntidadPadre>PersistenceMapper.java
```

```
└─ repository
   └─ SpringData<EntidadPadre>Repository.java
```

9. No inventes el mapeo: inspecciona tu tabla

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'TU_SCHEMA'
      AND table_name = 'TU_TABLA_PADRE'
ORDER BY ordinal_position;
```

Copia en tu cuaderno o README la lista de columnas reales. Tu clase JPA debe basarse en esa evidencia.

10. Plantilla para tu entidad de persistencia

```
@Entity
@Table(name = "TU_TABLA_PADRE", schema = "TU_SCHEMA")
public class TuEntidadPadreJpaEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "TU_PK")
    private Long id;

    @Column(name = "CAMPO_OBLIGATORIO", nullable = false)
    private String campoObligatorio;

    @Column(name = "CAMPO_UNICO", unique = true)
    private String campoUnico;

    protected TuEntidadPadreJpaEntity() {
    }

    // getters y setters necesarios para persistencia
}
```

Debes adaptar, no copiar

Si tu PK no es BIGINT/IDENTITY, si tus columnas tienen otros tipos, longitudes o nulabilidad, modifica el código según tu DDL. El profesor evaluará coherencia con tu base.

11. Crea tu repositorio Spring Data

```
public interface SpringDataTuEntidadPadreRepository
    extends JpaRepository<TuEntidadPadreJpaEntity, Long> {

    // Agrega métodos derivados solo si existen campos que lo justifiquen.
    // Ejemplo:
    // Optional<TuEntidadPadreJpaEntity> findByCodigo(String codigo);
}
```

12. Conserva separado tu modelo de dominio

Tu clase de dominio NO debe llevar @Entity, @Table ni @Column. Debe representar el concepto del negocio. Si ya la creaste en capítulos anteriores, ajústala solo si es necesario, pero no la conviertas en entidad JPA.

13. Crea un mapper de persistencia

```
public final class TuEntidadPadrePersistenceMapper {

    private TuEntidadPadrePersistenceMapper() {}

    public static TuEntidadPadre toDomain(TuEntidadPadreJpaEntity entity) {
        return new TuEntidadPadre(
            entity.getId()
            // completa el resto según tu dominio
        );
    }
}
```

14. Tu relación 1:N

Busca físicamente la FK en tu tabla dependiente. Si tu relación es equivalente a Cliente 1:N Vehículo, la FK debe estar en la tabla N. En JPA, la entidad dependiente normalmente puede declarar:

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "TU_FK", nullable = false)
private TuEntidadPadreJpaEntity padre;
```

No agregues una colección @OneToMany solo porque “JPA permite hacerlo”. Primero justifica si realmente necesitas navegar desde el padre hacia toda la colección.

15. Prueba que tu backend usa la base real

1. Arranca Spring Boot.
2. Verifica que Hibernate no reporte errores de validación.
3. Ejecuta tu endpoint de alta o lectura.
4. Observa el SQL generado en la consola de IntelliJ.
5. Abre DataGrip.
6. Ejecuta un SELECT sobre tu tabla y comprueba que el dato está allí.

```
SELECT *
FROM TU_SCHEMA.TU_TABLA_PADRE
ORDER BY TU_PK DESC;
```

16. Pruebas negativas obligatorias

Prueba	Qué debe ocurrir
Valor duplicado en una columna UNIQUE	PostgreSQL/JPA debe rechazar la operación.
Campo NOT NULL omitido	La operación debe fallar o ser validada antes.
Schema incorrecto en application.yml	La aplicación debe fallar al validar el mapeo.
Nombre de columna incorrecto en @Column	Hibernate debe detectar inconsistencia o fallar al operar.

17. Qué debes poder defender

- Por qué tu clase de dominio no tiene @Entity.
- Qué tabla mapea tu <EntidadPadre>JpaEntity.
- Cuál es la PK real y cómo se genera.
- Qué columnas son NOT NULL y cuáles UNIQUE.

- Qué diferencia existe entre JPA, Hibernate y Spring Data JPA.
- Por qué usas `ddl-auto=validate`.
- En qué tabla está la FK de tu relación 1:N.
- Cómo comprobaste que el dato quedó físicamente guardado.

18. Evidencia que debes entregar

- Captura o texto del `SELECT current_database()/current_schema()`.
- `application.yml` sin publicar credenciales reales.
- Entidad JPA de tu tabla padre.
- Repositorio Spring Data.
- Mapper a dominio.
- Consulta SQL que demuestra persistencia.
- Prueba HTTP ejecutada.
- Commit Git del capítulo.

19. Commit sugerido

```
git add .
git commit -m "feat: conectar backend con PostgreSQL y mapear entidad principal con JPA"
```

20. Definition of Done

- Mi backend inicia conectado a la base correcta.
- Hibernate valida mi schema sin crear tablas.
- Mi entidad JPA coincide con el DDL real.
- Mi dominio no depende de JPA.
- Tengo un repositorio Spring Data funcional.
- Puedo demostrar al menos una operación real contra PostgreSQL.
- Puedo explicar dónde está la FK de mi relación 1:N.
- Mi grupo dejó evidencia en Git y todos pueden defenderla.

21. Qué viene después

En el Capítulo 06 formalizaremos la arquitectura hexagonal: el caso de uso dejará de depender directamente del repositorio Spring Data y se introducirá un Port OUT implementado por un Persistence Adapter. Ahí comenzará a verse claramente la inversión de dependencias.